

REZUMAT

Aceasta lucrare prezinta trei tehnici fundamentale de **feature engineering** aplicabile oricarei serii temporale: **encoding ciclic**, **lag features** si **rolling features**. Datele brute (timestamp, valoare) NU sunt direct utilizabile de modelele clasice de Machine Learning - cele 3 tehnici le transforma in coloane informative. Subliniem o capcana tehnica subtila: rolling features fara `.shift(1)` introduc **data leakage** - feature-ul vede target-ul si modelul 'invata' raspunsul. Validarea empirica se realizeaza in cadrul lucrarii de disertatie prin aplicarea tehnicilor pe date din **domeniul energetic** (consum SUA, pret Spania, productie solara India), modelul XGBoost atingand $R^2 = 0.968$ in predictia pretului energetic spaniol.

PROBLEMA - DE CE FEATURE ENGINEERING?

Pentru ML clasic (XGBoost, RandomForest, regresie), datele de tip serie temporala trebuie transformate intr-un tabel cu coloane informative. Pattern-urile tipice ale seriilor temporale (frecvent intalnite in domeniul energetic, financiar, meteorologic) sunt: **sezonalitate orara** (consum mare seara, mic noaptea), **ciclu saptamanal** (fabricile lucreaza luni-vineri), **autoregresivitate** (pretul peste o ora $\approx$ pretul actual), **trend si volatilitate recenta**. Toate aceste pattern-uri sunt ASCUNSE in (timestamp, valoare) - trebuie extrase explicit. Solutia: cele 3 tehnici de mai jos.

CELE 3 TEHNICI

TEHNICA	PROBLEMA	SOLUTIA	FEATURE-URI CREATE
1. Encoding ciclic	Ora 23 si 0 sunt vecine pe ceas dar la distanta 23 in reprezentare liniara.	Asezam timpul pe cerc trigonometric (sin, cos).	hour_sin, hour_cos month_sin, month_cos dow_sin, dow_cos
2. Lag features	Modelul nu are 'memorie' a trecutului.	Adaugam coloane cu valoarea de acum 1h, 24h, 168h.	price_lag_1 price_lag_24 price_lag_168
3. Rolling features	Un singur punct din trecut poate fi atipic.	Medie / deviatie pe fereastra alunecatoare CU <code>.shift(1)</code> .	price_roll_mean_3 price_roll_mean_24 price_roll_std_24

ENCODING CICLIC: MATEMATICA

$$\text{hour_sin} = \sin(2\pi \cdot h / 24) \quad \text{hour_cos} = \cos(2\pi \cdot h / 24)$$

Cele 2 coloane (sin, cos) identifica UNIC fiecare ora ca un punct pe cercul trigonometric. Distanța euclidiană între ora 23 și ora 0 pe cerc devine 0.27 (mica și naturală), nu 23. Aceeași tehnică funcționează pentru zi-săptămânii, luna, ziua-din-an.

ATENȚIE: ROLLING FEATURES SI DATA LEAKAGE

Detaliul tehnic cel mai important al lucrării. Implementarea naivă a rolling features introduce data leakage - feature-ul include în calcul valoarea pe care vrem să o prezicem.

GRESIT (data leakage)	CORECT (cu shift)
<pre>df['roll_mean_3'] = \ df['pret'].rolling(3).mean()</pre>	<pre>df['roll_mean_3'] = \ df['pret'].shift(1).rolling(3).mean()</pre>
La momentul t, feature include valoarea t. R^2 fals = 0.94 Model 'învătat' răspunsul.	La momentul t, feature folosește DOAR t-1, t-2, t-3. R^2 cîstit = 0.75 Model valid în producție.

REZULTATE EMPIRICE - PREȚUL ENERGETIC SPANIA

Aplicând cele 3 tehnici de feature engineering (cu `.shift(1)` corect aplicat) pe setul ENTSOE (34.896 ore, 78 features), am antrenat 7 modele cu validare cronologică (TimeSeriesSplit, 5 folds):

MODEL	$R^2 \uparrow$	RMSE $\downarrow$ (EUR/MWh)	MAPE $\downarrow$	OBSERVAȚII
LinearRegression	0.947	2.64	3.49%	Baseline solid
Ridge	0.948	2.63	3.46%	Regularizare L2
Lasso	0.956	2.40	2.93%	Regularizare L1 - elimina features
RandomForest	0.963	2.22	2.71%	Ensemble nelinear
XGBoost	0.968	2.07	2.57%	CASTIGATOR
XGBoost + Optuna	0.966	2.12	2.65%	Tuning bayesian (5 trials demo)
LSTM (mod demo)	-1.12	20.66	60%	Date insuficiente în demo

Toate modelele clasice obțin R^2 peste 0.94 - dovada că cele 3 tehnici aplicate corect au creat features extrem de informative. Diferența între cel mai slab (LinearRegression) și cel mai bun (XGBoost) este **doar 0.02 în R^2** - ceea ce contează nu este selecția modelului, ci calitatea feature engineering-ului.

CONCLUZII

1. Feature engineering > selecția modelului. Pe time series, calitatea features-urilor face mai mult decît ajustarea fină a modelului. Toate cele 6 modele clasice obțin scoruri excelente dacă features-urile sunt construite corect.

2. Detaliile tehnice decid între cîstit și fals. Un simplu `.shift(1)` înainte de rolling este diferența între un model care arată bine pe test și esuează în producție, și un model care performează consistent. Această capcană este subestimată în tutoriale.

3. Tehnicile sunt universale. Encoding ciclic, lag și rolling features funcționează la fel pe consum (SUA), preț (Spania) și producție solară (India). Sunt un fundament metodologic care se transferă între domenii. În pasul următor al lucrării voi aplica același pipeline și pe sesiunea 3 (producția solară India).

Date utilizate: ENTSOE - operatorul european de transport energie (preț Spania); PJM Interconnection (consum SUA); Kaggle Solar Power Generation (producție India). **Cod:** Python (pandas, scikit-learn, XGBoost) - reproductibil în github.com/diana-nenu.

Contact: Diana Nenu · diana_nenu@yahoo.com · Universitatea Titu Maiorescu, Master DSAI